Week 11 new (from Week 7 easier)

Week 7: Polyphase Structures. Efficient implementations of multi-rate filters using polyphase decomposition.

Week 7: Polyphase Structures

Lecture 13: Polyphase Decomposition and Efficient Decimation *Oppenheim & Schafer: Chapter 4 (4.7)*

- **0:00 – 0:20 | The Polyphase Concept:** Decomposing an impulse response into M sub-sequences.
- Equation: $H(z) = \sum_{k=0}^{M-1} z^{-k} E_k(z^M)$.
- Visualizing how a single FIR filter splits into parallel branches.
- **0:20 – 0:55 | Efficient Decimation and Noble Identities:**
- The naive approach: Computing outputs that the downsampler immediately throws away.
- First Noble Identity: Commuting the downsampler and the filter.
- Moving the downsampler *before* the polyphase filters $E_k(z)$ so that all arithmetic runs at the lower sampling rate, achieving an M-fold computational savings.
- **0:55 – 1:15 | ML Context – Grouped Convolutions:** How polyphase parallelization relates conceptually to grouped convolutions in PyTorch and highly efficient edge-deployment architectures (e.g., MobileNet).

Lecture 14: Efficient Interpolation and Filter Banks *Oppenheim & Schafer: Chapter 4 (4.7, 4.8)*

- **0:00 – 0:35 | Efficient Interpolation:**
- Second Noble Identity: Commuting the upsampler and the filter.
- Moving the upsampler *after* the polyphase filters. Computing outputs without multiplying by the inserted zeros, achieving an L-fold computational savings.
- **0:35 – 0:55 | Two-Channel Filter Banks:** Introduction to Analysis and Synthesis banks. Splitting a signal into lowpass and highpass subbands, downsampling, and perfectly reconstructing them.
- **0:55 – 1:15 | ML Context – Learned Filter Banks:** Transitioning from fixed polyphase filter banks to learned filter banks in ML. Using fixed subband decompositions (like

wavelet scattering transforms or Mel-filter banks) as highly efficient, non-trainable feature extractors that feed into lightweight PyTorch classifiers.

Prompt: Can you create slides and python code for these two 75 minute classes of a DSP and ML course for a combined senior and first year Electrical and Computer Engineering class? The DSP textbook is Discrete-Time Signal Processing by Oppenheim and Schafer. The ML textbook is Machine Learning with PyTorch and Scikit-Learn by Raschka. Add some homework problems with solutions and a weekly quiz with solutions as well.

ChatGPT: (Medium)

Done. I created the complete Week 7 Polyphase Structures package.

Lecture slides

- Lecture 13 — Polyphase Decomposition and Efficient Decimation
- Lecture 14 — Efficient Interpolation and Filter Banks

The decks include pacing for the 75-minute classes, worked examples, Noble-identity diagrams, computational-savings examples, concept checks, instructor notes, ML connections, and exit questions.

A point I treated carefully is that the Noble identities do **not** allow an arbitrary filter to move through a rate changer. Lecture 13 explicitly distinguishes

$$G(z^M) \rightarrow\downarrow M$$

from an arbitrary $G(z)$, and Lecture 14 makes the analogous distinction for interpolation. This prevents a common conceptual error.

The ML sections also distinguish the useful analogy from an exact equivalence: polyphase decomposition partitions **time/tap phases modulo the rate-change factor**, whereas grouped convolution partitions **feature channels**.

Python material

- Jupyter notebook — Week 7 Polyphase DSP + ML Demos
- Plain Python version

The code includes:

- polyphase decomposition and reconstruction,
- naive versus polyphase decimation,
- numerical verification of the First Noble Identity,
- PyTorch grouped `Conv1d`,
- numerical verification of the Second Noble Identity,

- efficient interpolation with `scipy.signal.upfirdn`,
- a perfectly reconstructing Haar two-channel bank,
- fixed FIR subband features with Scikit-Learn,
- and a lightweight PyTorch classifier after a fixed DSP front end.

The notebook was executed successfully end-to-end.

Homework and weekly quiz

- Homework + Weekly Quiz + Complete Solutions

The packet contains eight homework problems with complete solutions and grading guidance, including:

- polyphase decomposition,
- operation-count savings,
- a Noble-identity counterexample,
- the efficient-decimator derivation,
- interpolation phases,
- Haar perfect reconstruction,
- two-channel perfect-reconstruction conditions,
- and DSP versus grouped/depthwise CNN architecture design.

It also contains an eight-question weekly quiz with solutions plus differentiation notes for the mixed first-year/senior cohort.

Complete package

- Download the entire Week 7 package

Week 7, Lecture 13

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs . .. for inline math and double dollar signs

. ..

for block/display math. Do not use (...) or [...]. Output the raw text without escaping the dollar signs.

Chat GPT (Medium):

Below is an instructor-oriented expansion of **Week 7, Lecture 13: Polyphase Decomposition and Efficient Decimation**. I'm following the organization and intent of your slide deck: polyphase decomposition → efficient decimation → First Noble Identity →

computational savings → analogy to grouped convolution. The central theme of the lecture is that polyphase processing **does not change the FIR filter or its retained outputs; it reorganizes the arithmetic so that samples which would later be discarded are never computed in the first place.**

Slide 1 — Polyphase Decomposition and Efficient Decimation

The most important opening idea is:

> **We are not designing a different filter. We are computing exactly the same desired decimated output more efficiently.**

Students already know the conventional decimator:

$$x[n] \rightarrow H(z) \rightarrow\downarrow M \rightarrow y[m]$$

The problem is computational waste. If the downsampler keeps only

$$v[0], v[M], v[2M], \ldots,$$

then all the filtered samples between those indices are computed and immediately thrown away.

For example, suppose $M = 4$. A conventional implementation computes

$$v[0], v[1], v[2], v[3], v[4], v[5], \ldots$$

but keeps only

$$v[0], v[4], v[8], v[12], \ldots$$

So a natural question is:

> Why calculate $v[1]$, $v[2]$, and $v[3]$ if the next block immediately discards them?

That question motivates the entire lecture.

Good student answer

> Polyphase decomposition reorganizes the FIR filter so that only computations contributing to the retained downsampled outputs are performed.

An even better answer adds:

> It is an exact rearrangement. It does not approximate $H(z)$ and does not change the output sequence $y[m]$.

Slide 2 — Roadmap and Learning Objectives

The lecture has two distinct intellectual tasks.

First, students need the **index bookkeeping** of polyphase decomposition:

$$e_k[r] = h[rM + k].$$

Second, they need to understand **why this algebra matters computationally**.

It is easy for students to learn the first part mechanically without understanding the second. Keep bringing the discussion back to:

> What calculation have we eliminated?

The ML section is intentionally an analogy rather than a mathematical equivalence. Your stated objective of distinguishing polyphase time-phase partitioning from grouped/channel convolution is important.

A useful framing is:

- Polyphase DSP partitions along **time/tap phase**.
- Grouped CNN convolution partitions along **channels**.
- Both exploit known structure to avoid a dense computation.

Slide 3 — The Polyphase Concept

The central question on this slide is excellent:

> If only every Mth filtered sample survives, why compute the other $M - 1$ outputs at all?

Before introducing equations, give students a concrete case.

Suppose

$$M = 3.$$

Imagine filtering generates

$$v[0], v[1], v[2], v[3], v[4], v[5], v[6], \ldots$$

Downsampling by 3 produces

$$y[m] = v[3m],$$

so

$$y[0] = v[0], \qquad y[1] = v[3], \qquad y[2] = v[6], \ldots$$

The outputs

$$v[1], v[2], v[4], v[5], \ldots$$

will never be observed.

Useful question to ask students

“Does the efficient structure save work by approximating those discarded samples?”

Good answer

> No. It saves work by never calculating samples that are not needed.

That distinction matters.

Slide 4 — Start with One Ordinary FIR Filter

The basic polyphase definition is

$$e_k[r] = h[rM + k], \qquad k = 0,1, \ldots, M-1.$$

This means:

> Group FIR coefficients according to their index modulo M.

This is probably the single most useful intuitive description.

For $M = 3$:

- phase 0 contains indices satisfying $n \bmod 3 = 0$,
- phase 1 contains indices satisfying $n \bmod 3 = 1$,
- phase 2 contains indices satisfying $n \bmod 3 = 2$.

Using your example

$$h = \{1,2,3,4,5,6,7,8\},$$

we obtain

$$e_0 = \{1,4,7\},$$

$$e_1 = \{2,5,8\},$$

and

$$e_2 = \{3,6\}.$$

A table makes this particularly clear:

Original index n	$h[n]$	$n \bmod 3$	Phase
0	1	0	e_0
1	2	1	e_1
2	3	2	e_2
3	4	0	e_0
4	5	1	e_1
5	6	2	e_2
6	7	0	e_0
7	8	1	e_1

Common mistake

Students often think “phase” means phase angle such as

$$\angle H(e^{j\omega}).$$

It does not.

Here, **polyphase phase means index class modulo M.**

That terminology is worth explicitly clarifying.

Slide 5 — Polyphase Decomposition in the z-Domain

The expression is

$$H(z) = \sum_{k=0}^{M-1} z^{-k}\, E_k(z^M).$$

The best derivation begins with

$$H(z) = \sum_{n} h\,[n] z^{-n}.$$

Every index n can be written uniquely as

$$n = rM + k,$$

where

$$k \in \{0,1,\dots,M-1\}.$$

Therefore,

$$H(z) = \sum_{k=0}^{M-1} \sum_{r} h\,[rM+k] z^{-(rM+k)}.$$

Separate the powers:

$$H(z) = \sum_{k=0}^{M-1} z^{-k} \sum_{r} h\,[rM+k] (z^{M})^{-r}.$$

Define

$$e_k[r] = h[rM+k]$$

and

$$E_k(z) = \sum_{r} e_k\,[r] z^{-r}.$$

Then

$$H(z) = \sum_{k=0}^{M-1} z^{-k}\, E_k(z^M).$$

That derivation is important because the structure does not appear arbitrary once students see the substitution

$$n = rM + k.$$

Your deck correctly identifies that substitution as the heart of the formula.

What does z^{-k} do?

A good verbal interpretation is:

> $E_k(z^M)$ contains the coefficients belonging to phase k, but z^{-k} restores their original offset inside the full FIR sequence.

For $M = 3$,

$$H(z) = E_0(z^3) + z^{-1}E_1(z^3) + z^{-2}E_2(z^3).$$

If

$$e_0 = \{1,4,7\},$$

then

$$E_0(z) = 1 + 4z^{-1} + 7z^{-2}.$$

Therefore,

$$E_0(z^3) = 1 + 4z^{-3} + 7z^{-6}.$$

Notice how those coefficients return to positions 0, 3, and 6.

Similarly,

$$z^{-1}E_1(z^3)$$

places phase-1 coefficients at positions

$$1,4,7, \ldots$$

That is a very useful concrete demonstration.

Slide 6 — Concept Check: $M = 4$

Question:

> For $M = 4$, which FIR taps belong to phase E_2?

Students should recognize the condition

$$n = 4r + 2.$$

Thus the indices are

$$2,6,10,14, \ldots$$

and

$$e_2[r] = h[4r + 2].$$

Weak answer

> Every fourth coefficient.

This is incomplete because it does not identify **which offset**.

Good answer

> E_2 contains coefficients whose indices are congruent to 2 modulo 4:
> $h[2], h[6], h[10], h[14], \ldots$.

Excellent answer

> Because phase k is defined by $n = rM + k$. For $M = 4$ and $k = 2$, this gives $n = 4r + 2$.

Slide 7 — Efficient Decimation and Noble Identities

This is where students should shift from:

> "Polyphase is an alternative representation of $H(z)$"

to

> "Polyphase exposes pieces of $H(z)$ that can be moved across the downsampler."

That is the computational payoff.

One useful verbal transition is:

> The z^M in $E_k(z^M)$ is not merely notation. It creates exactly the special filter structure required by the Noble Identity.

Slide 8 — The Naive Decimator Wastes Most FIR Outputs

The ordinary FIR output is

$$v[n] = \sum_k h\,[k]x[n-k].$$

After decimation,

$$y[m] = v[mM].$$

Therefore,

$$y[m] = \sum_k h\,[k]x[mM-k].$$

This equation reveals something crucial:

> We only need the FIR convolution evaluated at $n = mM$.

Suppose the filter has $N = 32$ taps and

$$M = 4.$$

A straightforward FIR implementation performs approximately

$$32F_s$$

multiplications per second.

But the output sampling rate is

$$\frac{F_s}{4}.$$

If we organize the algorithm so that roughly 32 multiplications occur only when an output is required, then the cost is

$$32\frac{F_s}{4} = 8F_s.$$

So we move from approximately

$$32F_s$$

to

$$8F_s$$

multiplications per second.

That is a factor-of-four savings.

Important conceptual point

Students sometimes say:

> "Polyphase reduces a 32-tap filter to an 8-tap filter."

That is misleading.

There are still 32 coefficients.

They are distributed among four branches of approximately eight coefficients each.

Slide 9 — First Noble Identity

This is probably the slide where precision matters most.

The identity is

$$G(z^M) \rightarrow\downarrow M$$

equivalent to

$$\downarrow M \rightarrow G(z).$$

The key condition is the special structure

$$G(z^M).$$

You cannot say:

$$G(z) \to\downarrow M \equiv\downarrow M \to G(z)$$

for an arbitrary $G(z)$.

That is false.

Why?

Consider

$$G(z) = 1 + z^{-1}.$$

Filtering first gives

$$v[n] = x[n] + x[n-1].$$

Then downsampling gives

$$y[m] = x[mM] + x[mM-1].$$

If instead we downsample first,

$$w[m] = x[mM],$$

then filter with

$$G(z) = 1 + z^{-1},$$

giving

$$\tilde{y}[m] = w[m] + w[m-1] = x[mM] + x[(m-1)M].$$

Generally,

$$x[mM-1] \neq x[(m-1)M].$$

So

$$y[m] \neq \tilde{y}[m].$$

This is an excellent counterexample.

Why does $G(z^M)$ work?

Suppose

$$G(z^M) = 1 + z^{-M}.$$

Filtering first gives

$$v[n] = x[n] + x[n-M].$$

Downsampling:

$$y[m] = x[mM] + x[(m-1)M].$$

Now downsample first:

$$w[m] = x[mM].$$

Filter at the lower rate with

$$G(z) = 1 + z^{-1},$$

giving

$$y[m] = w[m] + w[m-1] = x[mM] + x[(m-1)M].$$

Now they match exactly.

That example makes the Noble Identity much less abstract.

Slide 10 — Efficient Polyphase Decimator

The retained output can be expressed as

$$y[m] = \sum_k \sum_r e_k[r] x[(m-r)M - k].$$

The indexing looks complicated, but the conceptual structure is simple.

Each branch handles one residue class of the FIR coefficients.

For example, with $M = 3$:

- branch 0 handles $h[0], h[3], h[6], \ldots$,
- branch 1 handles $h[1], h[4], h[7], \ldots$,
- branch 2 handles $h[2], h[5], h[8], \ldots$.

The matching input samples must be offset accordingly.

This is why the warning in your instructor note matters:

> Do not imply that every branch simply receives `x[::M]`.

The branches correspond to different input phases.

For $M = 3$, the relevant input subsequences are conceptually something like

$$x[3m],$$

$$x[3m-1],$$

and

$$x[3m-2],$$

subject to implementation alignment and delay conventions.

Useful commutator picture

You can describe the input stream as being distributed cyclically:

- sample 0 → branch 0
- sample 1 → branch 1
- sample 2 → branch 2
- sample 3 → branch 0
- sample 4 → branch 1
- sample 5 → branch 2
- …

Each branch therefore sees samples at rate

$$\frac{F_s}{M}.$$

That makes the low-rate arithmetic intuitive.

Slide 11 — Where the M-Fold Savings Comes From

Your slide states the crucial distinction:

> The number of multiplies per retained output is not smaller; the number of unnecessary outputs is.

That is exactly what students should remember.

Naive cost:

$$C_{\text{naive}} \approx NF_s.$$

Polyphase cost:

$$C_{\text{polyphase}} \approx N\frac{F_s}{M}.$$

Equivalently,

$$C_{\text{polyphase}} \approx \frac{N}{M} F_s.$$

Therefore,

$$\frac{C_{\text{naive}}}{C_{\text{polyphase}}} \approx M.$$

Example

Suppose

$$N = 120, \qquad M = 5, \qquad F_s = 100\ \text{kHz}.$$

Naive:

$$120(100000) = 12{,}000{,}000$$

multiplications/s.

Polyphase:

$$120\left(\frac{100000}{5}\right) = 2{,}400{,}000$$

multiplications/s.

Savings:

$$\frac{12{,}000{,}000}{2{,}400{,}000} = 5.$$

So approximately a 5× improvement.

Slide 12 — Concept Check: $N = 48, M = 6$

Question:

> Roughly how many FIR multiplications per input sample are required?

The average is

$$\frac{N}{M} = \frac{48}{6} = 8.$$

Good answer

> About 8 multiplies per input sample on average, instead of 48.

Also acceptable

> Approximately a 6× reduction in arithmetic.

Best answer

> Each retained output still requires roughly 48 tap multiplications, but outputs occur only once every six input samples. Therefore the average computational cost is $48/6 = 8$ multiplies per input sample.

That answer demonstrates actual understanding.

Slide 13 — ML Context: Grouped Convolutions

This transition should be handled carefully.

The similarity is structural:

> large computation → structured branches → smaller independent computations → recombine.

But polyphase and grouped convolution partition different things.

A good sentence is:

> Polyphase decomposition exploits known time-rate structure; grouped convolution imposes structured channel connectivity.

Do not tell students:

> “Grouped convolution is the ML version of polyphase filtering.”

That goes too far.

Better:

> “Grouped convolution illustrates the same broad computational principle: use structure to avoid unnecessary dense computation.”

Your deck explicitly frames it this way.

Slide 14 — Grouped `Conv1d`: Channel Partitioning

For ordinary convolution with

```
torch.nn.Conv1d(
    in_channels=C_in,
    out_channels=C_out,
    kernel_size=K
)
```

each output channel can combine information from **all** input channels.

Suppose

$$C_{\text{in}} = 8, \qquad C_{\text{out}} = 16, \qquad K = 5.$$

Ignoring biases, the parameter count is

$$16 \times 8 \times 5 = 640.$$

Now suppose

$$\text{groups} = 4.$$

Each group sees only

$$\frac{8}{4} = 2$$

input channels and produces

$$\frac{16}{4} = 4$$

output channels.

Parameter count becomes

$$16 \times \frac{8}{4} \times 5 = 160.$$

So the number of weights drops from

$$640$$

to

$$160.$$

That is a 4× reduction.

Depthwise convolution

If

$$G = C_{\text{in}},$$

then each channel is filtered separately.

For

$$C_{\text{in}} = 8$$

and

$$K = 5,$$

a depthwise stage needs only approximately

$$8 \times 5 = 40$$

filter coefficients, before any later pointwise channel mixing.

This is the idea behind depthwise-separable architectures.

Slide 15 — Polyphase vs. Grouped Convolution

A useful comparison table is:

	Polyphase DSP	Grouped CNN
What is partitioned?	sample/tap phases	feature channels
Partition determined by	rate factor M	architecture choice G
Coefficients	original FIR coefficients	learned weights
Goal	exact efficient multirate implementation	reduce connectivity and computation
Mathematical equivalence?	exact rearrangement	generally changes architecture
Branch rate	often lower sampling rate	same feature-map sampling grid

Strong student answer to “How are they similar?”

> Both replace a dense computation with structured parallel branches.

Strong answer to “How are they different?”

> Polyphase partitions temporal/tap indices modulo the rate-change factor and preserves exact multirate behavior. Grouped convolution partitions learned feature channels and intentionally restricts network connectivity.

That distinction closely matches your slide.

Slide 16 — Coding Demo: Prove the Equivalences

This is a good place to turn abstract algebra into numerical evidence.

Demo 1 — Polyphase decomposition and reconstruction

Start with

```
h = np.arange(1, 9)
M = 3
```

Then

```
e0 = h[0::3]
e1 = h[1::3]
e2 = h[2::3]
```

Students should predict:

```
e0 = [1, 4, 7]
e1 = [2, 5, 8]
e2 = [3, 6]
```

Then reconstruct by interleaving and verify exact equality.

The question to ask is:

> Did we design three new filters?

Good answer:

> No. We partitioned the coefficients of one original filter.

Demo 2 — Naive decimation versus efficient implementation

Compute

```
v = scipy.signal.lfilter(h, 1, x)
y_naive = v[::M]
```

and compare it against an appropriate polyphase/downsample implementation such as `upfirdn`.

Ask before execution:

> Should these sequences be approximately equal or exactly equal apart from numerical precision and boundary conventions?

Good answer:

They should represent the same mathematical output, with differences only from numerical precision or alignment/boundary conventions.

Demo 3 — Noble Identity

Choose

$$G(z) = 1 + 0.5z^{-1} + 0.2z^{-2}.$$

Construct

$$G(z^3) = 1 + 0.5z^{-3} + 0.2z^{-6}.$$

Then compare

$$x[n] \to G(z^3) \to\downarrow 3$$

with

$$x[n] \to\downarrow 3 \to G(z).$$

They should match.

Then deliberately replace $G(z^3)$ with arbitrary $G(z)$.

They should no longer match.

That makes the restriction memorable.

Slide 17 — Wrap-Up

The strongest conceptual summary is:

1. Polyphase is an exact decomposition

$$H(z) = \sum_{k=0}^{M-1} z^{-k}\, E_k(z^M).$$

2. The coefficients are grouped modulo M

$$e_k[r] = h[rM + k].$$

3. Decimation creates wasted computation

The naive system calculates M times as many filtered output samples as it keeps.

4. The Noble Identity moves appropriately structured filtering to the low-rate side

$$G(z^M) \rightarrow\downarrow M \equiv\downarrow M \rightarrow G(z).$$

5. The ideal computation reduction is roughly

$$M.$$

6. Grouped convolution shares the broad principle of structured computation, but it is not mathematically the same operation.

That summary matches the conclusions of the deck.

Exit Question

Why is it incorrect to say, “A downsampler can always be moved before a filter”?

Weak answer

Because the outputs might change.

True, but not very informative.

Good answer

A filter and a downsampler do not generally commute.

Very good answer

The First Noble Identity permits moving a filter across a downsampler only when the high-rate filter has the special form $G(z^M)$. An arbitrary $G(z)$ does not satisfy the identity.

Excellent answer

With $G(z^M)$, the nonzero filter delays occur in multiples of M, so after downsampling they become ordinary unit delays in $G(z)$. An arbitrary filter contains delays that do not line up with the retained samples, so moving the downsampler changes which input samples contribute to the output.

That last response demonstrates that the student understands **why** the identity works rather than simply memorizing it.

Additional Concept Questions I Would Ask During the Lecture

Question 1

If

$$M = 5,$$

which coefficients belong to phase E_3?

Good answer:

$$h[3], h[8], h[13], h[18], \ldots$$

because

$$n = 5r + 3.$$

Question 2

Does polyphase decomposition reduce the number of FIR coefficients?

Good answer:

> No. It reorganizes the same coefficients among M branches.

Question 3

For a 60-tap filter and $M = 5$, about how many taps does each polyphase branch contain?

Good answer:

$$\frac{60}{5} = 12.$$

Approximately 12 taps per phase.

Question 4

If each branch has roughly N/M taps, why isn't the total computational saving only due to shorter filters?

Good answer:

> Because there are M branches. Collectively they still contain about N coefficients. The saving occurs because those branch filters operate at the lower rate F_s/M.

This is one of the best questions for detecting superficial understanding.

Question 5

Why does polyphase decomposition fit decimation so naturally?

Good answer:

> Downsampling selects samples according to their time index modulo M, while polyphase decomposition partitions filter coefficients according to index modulo M. Those two structures align.

Question 6

Is grouped `Conv1d` an implementation of a polyphase filter?

Good answer:

> No. It is only an analogy in structured computation. Polyphase partitions temporal/tap phases to implement an exact rate-changing system efficiently, whereas grouped convolution partitions feature channels and changes the network's connectivity.

The Three Ideas Students Most Need to Leave With

If students remember only three things from this lecture, I would make them these:

First: Polyphase is not a new filter.

$$H(z) = \sum_{k=0}^{M-1} z^{-k}\, E_k(z^M)$$

is simply another way to represent exactly the same $H(z)$.

Second: The computational saving comes from refusing to calculate outputs that decimation will throw away.

It is primarily a **rate-of-computation argument**, not a reduction in the total number of coefficients.

Third: The Noble Identity has a condition.

$$G(z^M) \rightarrow\downarrow M \equiv\downarrow M \rightarrow G(z)$$

does **not** imply that an arbitrary filter can be moved across a downsampler.

That last misconception is probably the most important one to catch before students leave Lecture 13.

Week 7, Lecture 14

Explain in more detail what's going on here and what good answers and/or examples are. For all mathematical equations, use standard single dollar signs . .. for inline math and double dollar signs

. ..

for block/display math. Do not use (...) or [...]. Output the raw text without escaping the dollar signs.

Chat GPT (Medium):

Below is an instructor-facing expansion of **Week 7, Lecture 14: Efficient Interpolation and Filter Banks**. I'm following the deck's structure and terminology closely. The lecture builds directly on the previous polyphase-decimation lecture, but now the computational waste comes from **multiplying inserted zeros** rather than **computing outputs that will later be discarded**. It then extends that multirate viewpoint into two-channel analysis/synthesis filter banks and finally into fixed versus learned subband front ends for machine learning.

Slide 1 — Efficient Interpolation and Filter Banks

The opening idea should be:

> In interpolation, we already know that $L-1$ out of every L samples after zero insertion are exactly zero. Why should a full-rate FIR filter spend multiplications on values we know are zero?

The conventional interpolator is

$$x[n] \rightarrow\uparrow L \rightarrow H(z) \rightarrow y[n].$$

The upsampler creates

$$v[n] = \begin{cases} x[n/L], & n = 0, \pm L, \pm 2L, \ldots \\ 0, & \text{otherwise.} \end{cases}$$

So for $L = 4$,

$$v[n] = \{x[0],0,0,0, x[1],0,0,0, x[2], \dots\}.$$

Then the FIR filter runs at the new rate

$$4F_s.$$

That means most multiplications involve zero-valued inputs.

The central computational question is:

> Can we compute the nonzero contributions at the original low rate and then place them directly into their correct high-rate output phases?

The answer is yes, using the same polyphase decomposition introduced for decimation.

A strong opening connection is:

- decimation: avoid computing outputs that are discarded,
- interpolation: avoid multiplying input samples that were deliberately inserted as zeros.

That distinction becomes useful again in the exit question.

Slide 2 — Roadmap and Learning Objectives

This lecture has three conceptual layers.

Layer 1: Efficient interpolation

Students already know the mathematical rate-change operation. Now they should understand how polyphase reorganizes it computationally.

The central identity is

$$H(z) = \sum_{k=0}^{L-1} z^{-k} E_k(z^L).$$

Layer 2: Filter banks

Once students see several polyphase branches operating in parallel, moving to a bank of frequency-selective branches becomes natural:

$$x[n] \to \begin{cases} H_0(z) \\ H_1(z) \end{cases} \to \begin{cases} \downarrow 2 \\ \downarrow 2 \end{cases}.$$

Layer 3: ML front ends

The filter bank becomes a feature extractor.

That creates a natural conceptual progression:

$$\text{signal} \rightarrow \text{subbands} \rightarrow \text{features} \rightarrow \text{classifier}.$$

The deck explicitly frames the lecture around these three stages: efficient interpolation, two-channel filter banks, and fixed/learned subband front ends.

Slide 3 — Efficient Interpolation

The key idea here is:

> An interpolation factor L creates L distinct output phases.

For $L = 3$, the output samples can be divided into three classes:

$$y[3m],$$

$$y[3m + 1],$$

and

$$y[3m + 2].$$

That is the exact same modulo-class idea that appeared in polyphase decimation.

Instead of thinking:

> "Upsample, create zeros, then filter everything,"

students should start thinking:

> "Compute each output phase directly."

That is the mental shift.

A good analogy is a three-lane conveyor belt:

- branch 0 generates outputs at indices $0,3,6,$ ...,
- branch 1 generates outputs at indices $1,4,7,$ ...,
- branch 2 generates outputs at indices $2,5,8,$ ....

The commutator then interleaves those three low-rate streams.

Slide 4 — Why Naive Interpolation Wastes Arithmetic

Suppose

$$L = 4.$$

Zero insertion produces

$$v[0] = x[0],$$
$$v[1] = 0,$$
$$v[2] = 0,$$
$$v[3] = 0,$$
$$v[4] = x[1],$$

and so on.

Now consider an FIR filter

$$y[n] = \sum_{k} h\,[k]v[n-k].$$

Many terms in each convolution sum multiply coefficients by zeros.

For example, suppose

$$h = \{h[0], h[1], h[2], h[3], h[4], \dots\}.$$

At one output phase, only taps whose indices line up with actual nonzero upsampled samples contribute.

The rest are guaranteed zero.

That is exactly why the decomposition

$$H(z) = \sum_{k=0}^{L-1} z^{-k}\, E_k(z^L)$$

is useful.

Each polyphase component captures the coefficients relevant to one output phase.

Your slide states this nicely: each output phase is a low-rate convolution of $x[n]$ with one phase $E_k(z)$.

Good student explanation

> The zero-stuffed signal is sparse. Polyphase decomposition organizes the filter so each branch computes only products that can actually be nonzero.

Better answer

> Each branch produces one output residue class modulo L, so there is no need to construct the zero-stuffed sequence explicitly.

Slide 5 — Second Noble Identity

This is the interpolation counterpart of the First Noble Identity.

The valid equivalence is

$$\uparrow L \rightarrow G(z^L)$$

equivalent to

$$G(z) \rightarrow \uparrow L.$$

The important point is again the special structure:

$$G(z^L).$$

An arbitrary $G(z)$ cannot simply be moved through an upsampler.

This condition deserves strong emphasis.

A concrete example

Let

$$G(z) = 1 + az^{-1}.$$

Then

$$G(z^L) = 1 + az^{-L}.$$

Suppose $L = 3$.

Then

$$G(z^3) = 1 + az^{-3}.$$

On the left structure:

$$x[n] \rightarrow \uparrow 3 \rightarrow G(z^3),$$

the filter operates on the high-rate sequence but only accesses samples separated by 3.

Those samples correspond exactly to adjacent samples of the original low-rate $x[n]$.

So the same computation can be performed first as

$$w[n] = x[n] + ax[n-1]$$

at the low rate, followed by $\uparrow 3$.

Thus

$$\uparrow 3 \rightarrow G(z^3) \equiv G(z) \rightarrow \uparrow 3.$$

Bad oversimplification

> You can move filters across upsamplers.

Good answer

> You can move a filter across an upsampler only when the high-rate filter has the form $G(z^L)$.

Excellent answer

> The delays in $G(z^L)$ occur in multiples of L, so after moving to the low-rate side those delays become ordinary unit delays in $G(z)$.

Your slide explicitly warns that an arbitrary filter does not commute with $\uparrow L$.

Slide 6 — Polyphase Interpolator

This slide contains one of the most intuitive equations in the lecture:

$$y[mL + k] = (e_k * x)[m].$$

This deserves to be unpacked carefully.

For a given phase k:

$$k = 0,1, \dots, L-1,$$

the branch $E_k(z)$ computes all output samples whose index satisfies

$$n \bmod L = k.$$

For example, with $L = 4$:

Phase 0

$$y[4m] = (e_0 * x)[m].$$

This produces

$$y[0], y[4], y[8], \ldots$$

Phase 1

$$y[4m + 1] = (e_1 * x)[m].$$

This produces

$$y[1], y[5], y[9], \ldots$$

Phase 2

$$y[4m + 2] = (e_2 * x)[m].$$

Phase 3

$$y[4m + 3] = (e_3 * x)[m].$$

Then the commutator interleaves these branch outputs:

$$\{y_0[0], y_1[0], y_2[0], y_3[0], y_0[1], y_1[1], \ldots\}.$$

That is the high-rate output.

Key teaching point

The branches do not create redundant copies of the output.

Each branch is responsible for a different set of output indices.

Good student answer to "What does phase k produce?"

> All output samples whose indices are congruent to k modulo L.

That is the cleanest answer.

Your deck emphasizes exactly this interpretation.

Slide 7 — L-Fold Savings and Exact Equivalence

Suppose the interpolation filter contains N taps.

Naively:

- zero-stuffed signal rate is LF_s,
- FIR filter runs at LF_s,

- cost is approximately

$$NLF_s$$

multiplications per second.

With polyphase interpolation:

- the N coefficients are split among L branches,
- each branch has roughly N/L taps,
- each branch operates at F_s,
- there are L branches.

So total cost is approximately

$$L\left(\frac{N}{L}F_s\right) = NF_s.$$

Therefore the ideal speedup is

$$\frac{NLF_s}{NF_s} = L.$$

Very important nuance

The filter itself has not become simpler.

The same interpolation filter requirements still apply.

You still need the appropriate anti-imaging low-pass response and often an interpolation gain of L.

The slide explicitly notes that efficiency does not replace interpolation-filter design.

Example

Suppose

$$N = 48, \qquad L = 4, \qquad F_s = 10\ \text{kHz}.$$

Naive:

$$48 \cdot 4 \cdot 10000 = 1.92 \times 10^6$$

multiplies/s.

Polyphase:

$$48 \cdot 10000 = 4.8 \times 10^5$$

multiplies/s.

Ideal saving:

$$4\times.$$

Slide 8 — Concept Check

Question:

> A 36-tap FIR interpolates by $L = 4$. What ideal arithmetic saving should polyphase give?

Minimal acceptable answer

> About 4×.

Better answer

> The naive implementation requires about $36 \cdot 4F_s$ multiplications per second, while polyphase requires about $36F_s$.

Excellent answer

> Polyphase does not eliminate coefficients. It reorganizes the 36 coefficients into four phases that operate at the low rate, avoiding the zero-valued products created by upsampling.

That explanation is more valuable than the number itself.

Slide 9 — Two-Channel Filter Banks

This marks a conceptual change.

Before this point, there is one signal path and one rate conversion.

Now there are parallel subband paths:

$$x[n] \rightarrow \begin{matrix} H_0(z) \\ H_1(z) \end{matrix}.$$

For a two-channel bank:

- $H_0(z)$ is typically lowpass,
- $H_1(z)$ is typically highpass.

After filtering, each branch is downsampled by 2.

So the bank separates the input signal into two frequency regions and represents each at half the original rate.

This is why the deck says:

> Split the signal into subbands, reduce each rate, then combine them again.

That is the right conceptual summary.

Slide 10 — Analysis Bank

The analysis bank is

$$x[n] \to H_0(z) \to\downarrow 2$$

and

$$x[n] \to H_1(z) \to\downarrow 2.$$

The outputs are the low and high subbands.

Why can we downsample each branch?

Because the filtering reduces the bandwidth in each branch.

A lowpass branch might retain approximately

$$0 \le |\omega| \le \frac{\pi}{2},$$

while a highpass branch covers the complementary upper half.

Once each branch occupies only about half the original bandwidth, downsampling by 2 becomes plausible.

Critical sampling

The original sequence contains

$$F_s$$

samples per second.

After splitting:

- low branch: $F_s/2$,
- high branch: $F_s/2$.

Total:

$$\frac{F_s}{2} + \frac{F_s}{2} = F_s.$$

Thus the representation is **critically sampled**.

There is no increase in the total number of samples per second across all channels.

Good student explanation

> Each branch contains a smaller spectral region, so each can be represented at a lower sampling rate.

Important caution

The filters must suppress out-of-band energy before downsampling.

Otherwise each branch aliases.

The deck emphasizes this connection back to ordinary decimation.

Audio Example for the Analysis Bank

Suppose an audio signal contains:

- bass at 200 Hz,
- speech components around 1 kHz,
- cymbal transients around 6 kHz.

A two-band analysis bank might separate roughly:

- low frequencies,
- high frequencies.

The low branch might emphasize:

- pitch,
- slowly varying spectral envelope,
- low-frequency energy.

The high branch might emphasize:

- transients,
- consonants,
- edges,
- fine temporal detail.

That is why filter banks are useful in compression, denoising, and feature extraction.

Slide 11 — Synthesis Bank

The synthesis bank reverses the process:

$$\text{low subband} \rightarrow \uparrow 2 \rightarrow F_0(z)$$

and

$$\text{high subband} \rightarrow \uparrow 2 \rightarrow F_1(z),$$

followed by summation.

The target is

$$\hat{x}[n] = c\,x[n-d].$$

That is considered perfect reconstruction because:

- c is just constant gain,
- d is just delay.

Neither changes the essential waveform shape.

Good student answer to “What does perfect reconstruction mean?”

> The reconstructed signal is identical to the original except possibly for a constant scale factor and delay.

Weak answer

> The output looks approximately like the input.

That is not strong enough.

Perfect reconstruction is an exact structural condition.

Slide 12 — Perfect Reconstruction Conditions

There are two simultaneous requirements.

1. Alias cancellation

The alias term must satisfy

$$F_0(z)H_0(-z) + F_1(z)H_1(-z) = 0.$$

2. Desired-path condition

The remaining signal transfer function must satisfy

$$\frac{1}{2}[F_0(z)H_0(z) + F_1(z)H_1(z)] = cz^{-d}.$$

These correspond to two different problems.

Alias problem

Downsampling causes the spectral copies of each branch to fold.

So individually, branch outputs contain alias terms.

Perfect-reconstruction banks are designed so that those alias terms cancel when the synthesis branches are summed.

Distortion problem

Even after aliasing is canceled, the desired signal component could still be distorted.

For example, the amplitude response might vary with frequency.

Perfect reconstruction requires the surviving transfer function to be only

$$cz^{-d}.$$

That is flat magnitude with linear phase corresponding only to delay.

The deck explicitly distinguishes these two requirements.

A Useful Way to Explain Perfect Reconstruction

Students often think:

> "If both subbands are approximately correct, reconstruction should be approximately correct."

That is not the full story.

The synthesis bank relies on **joint cancellation**.

Each branch alone may contain terms that are undesirable.

But after combining the branches, those terms can cancel.

That is a useful systems idea:

> Individual paths do not need to be perfect; the combined system must satisfy the reconstruction conditions.

Slide 13 — Haar Bank

This is one of the best examples in the lecture because it makes perfect reconstruction completely transparent.

For pairs of samples,

$$x[2m]$$

and

$$x[2m+1],$$

define

$$a[m] = \frac{x[2m] + x[2m+1]}{\sqrt{2}},$$

and

$$d[m] = \frac{x[2m] - x[2m+1]}{\sqrt{2}}.$$

Interpretation:

- $a[m]$ is an average-like low-frequency quantity,
- $d[m]$ is a difference-like high-frequency quantity.

Now reconstruct:

$$x[2m] = \frac{a[m] + d[m]}{\sqrt{2}},$$

$$x[2m+1] = \frac{a[m] - d[m]}{\sqrt{2}}.$$

Derivation

Add $a[m]$ and $d[m]$:

$$a[m] + d[m] = \frac{x[2m] + x[2m+1] + x[2m] - x[2m+1]}{\sqrt{2}}.$$

So

$$a[m] + d[m] = \sqrt{2}x[2m].$$

Therefore,

$$x[2m] = \frac{a[m] + d[m]}{\sqrt{2}}.$$

Similarly,

$$a[m] - d[m] = \sqrt{2}x[2m + 1].$$

So reconstruction is exact.

Nice numerical example

Suppose

$$x[0] = 3, \qquad x[1] = 1.$$

Then

$$a[0] = \frac{3 + 1}{\sqrt{2}} = 2\sqrt{2},$$

and

$$d[0] = \frac{3 - 1}{\sqrt{2}} = \sqrt{2}.$$

Reconstruct:

$$x[0] = \frac{2\sqrt{2} + \sqrt{2}}{\sqrt{2}} = 3,$$

$$x[1] = \frac{2\sqrt{2} - \sqrt{2}}{\sqrt{2}} = 1.$$

That is perfect reconstruction in its simplest form.

The deck also notes that the Haar transform is orthonormal and preserves energy.

Energy Preservation in the Haar Bank

This is a good enrichment point for senior students.

For one pair,

$$a^2[m] + d^2[m] = x^2[2m] + x^2[2m+1].$$

Proof:

$$a^2 = \frac{(x_0 + x_1)^2}{2},$$

$$d^2 = \frac{(x_0 - x_1)^2}{2}.$$

Add them:

$$a^2 + d^2 = \frac{x_0^2 + 2x_0x_1 + x_1^2 + x_0^2 - 2x_0x_1 + x_1^2}{2}.$$

Therefore,

$$a^2 + d^2 = x_0^2 + x_1^2.$$

So the transform redistributes energy but does not change total energy.

Slide 14 — ML Context: Fixed and Learned Filter Banks

The conceptual bridge is:

> A filter bank can be viewed as a feature extractor.

Instead of trying to reconstruct the waveform, ML may use the subband outputs as features.

So the pipeline becomes

$$x[n] \rightarrow \text{filter bank} \rightarrow \text{subband features} \rightarrow \text{classifier}.$$

At this point students should distinguish two objectives:

Classical DSP filter bank

Often cares about:

- reconstruction,
- alias cancellation,
- known frequency partitions.

ML front end

Often cares about:

- discriminative information,
- robustness,
- compact feature representation,
- computational efficiency.

Perfect reconstruction may not even be required.

Slide 15 — Fixed Subband Features

Examples in your slide include:

- Mel filter-bank energies,
- wavelet coefficients,
- scattering-like features,
- hand-designed frequency bands.

The key advantage is strong prior structure.

A fixed bank says:

> We already know something useful about how signals should be partitioned.

For example, suppose we classify machines based on vibration data.

You might define four bands:

$$B_1: 0 - 100 \text{ Hz},$$

$$B_2: 100 - 300 \text{ Hz},$$

$$B_3: 300 - 800 \text{ Hz},$$

$$B_4: 800 - 2000 \text{ Hz}.$$

For each band, calculate energy

$$E_k = \sum_n y_k^2 [n].$$

Then use

$$\mathbf{f} = [\log E_1, \log E_2, \log E_3, \log E_4]$$

as a feature vector.

Now a classifier only sees four numbers instead of hundreds or thousands of waveform samples.

Good explanation of why this helps

> The DSP front end converts a high-dimensional waveform into physically meaningful low-dimensional features, so the learned model may require fewer parameters and less training data.

Important nuance

The deck correctly warns that Mel filter banks are often applied to magnitude or power spectra, not necessarily implemented as time-domain perfect-reconstruction banks.

That is worth saying explicitly.

Slide 16 — Learned Filter Banks

A learned `Conv1d` layer can act like an analysis bank.

Suppose there are K kernels:

$$h_0[n], h_1[n], \dots, h_{K-1}[n].$$

Each produces

$$y_k[n] = (h_k * x)[n].$$

The channels

$$y_0[n], y_1[n], \dots, y_{K-1}[n]$$

are analogous to subbands.

But unlike a classical filter bank, the filters are learned from the task.

Fixed bank

Advantages:

- interpretable,
- predictable,
- stable,
- few trainable parameters,
- often data-efficient.

Disadvantages:

- may not match the exact classification task.

Learned bank

Advantages:

- task-adaptive,
- can discover useful representations.

Disadvantages:

- requires data,
- optimization,
- regularization,
- may learn less interpretable responses.

Hybrid

A particularly useful engineering strategy is:

> Initialize a learned filter bank from DSP-designed filters, then fine-tune.

That combines prior knowledge with adaptability.

Important ML Misconception

A `Conv1d` layer with stride 2 is not automatically a well-designed multirate filter bank.

Stride 2 performs rate reduction.

Therefore, anti-aliasing still matters.

If a learned filter does not sufficiently suppress high-frequency content before downsampling, aliasing may occur.

A strong student should be able to say:

> Learned downsampling does not repeal sampling theory.

That is exactly the connection your deck makes back to the previous multirate material.

Another Important Misconception

Do not let students assume:

> Learned convolutional filters automatically satisfy perfect reconstruction.

They do not.

Perfect reconstruction requires constraints on the analysis and synthesis filters.

An unconstrained neural network may learn useful features while having no inverse at all.

That is perfectly acceptable for classification, but it is not a PR filter bank.

Slide 17 — Four Fixed Bands + Tiny Classifier

This is an excellent demonstration because it shows feature engineering concretely.

Suppose you generate three classes:

Class A

Dominant energy around

$$200 \text{ Hz}.$$

Class B

Dominant energy around

$$700 \text{ Hz}.$$

Class C

Dominant energy around

$$1500 \text{ Hz}.$$

Now construct four fixed FIR bands.

For each signal, calculate

$$E_k = \sum_n y_k^2 [n].$$

Use features

$$f_k = \log(E_k + \epsilon).$$

The feature vector becomes

$$\mathbf{f} = [f_0, f_1, f_2, f_3].$$

If each class has energy concentrated in a different band, then the data may look something like:

- Class A: $[8.2, 1.1, 0.3, 0.2]$
- Class B: $[0.8, 7.6, 1.0, 0.2]$
- Class C: $[0.2, 0.5, 1.4, 8.1]$

That can be almost trivially separable by logistic regression.

This makes the teaching point very clear:

> Good feature engineering can turn a difficult raw-signal classification problem into an easy low-dimensional classification problem.

Your slide makes exactly that point: complexity can be moved from the learned model into a known signal-processing transform.

Good Discussion Question for the Demo

Ask:

> Suppose the class frequencies move substantially after deployment. Would the fixed filter bank still be optimal?

Good answer

> Not necessarily. A fixed bank reflects assumptions about where useful information lies. If the signal statistics shift, the features may become less discriminative.

Better answer

> A learned or hybrid front end could adapt its frequency responses, whereas the fixed bank cannot unless we redesign it.

That gets students to think about the bias-versus-adaptability tradeoff.

Slide 18 — Wrap-Up

The lecture can be summarized around five ideas.

1. Polyphase interpolation avoids zero multiplies

Rather than explicitly producing the sparse upsampled sequence, compute each output phase directly.

2. The Second Noble Identity permits low-rate filtering

$$\uparrow L \rightarrow G(z^L) \equiv G(z) \rightarrow \uparrow L.$$

But only for the required z^L structure.

3. Two-channel filter banks split and recombine signals

Analysis:

$$x[n] \rightarrow \text{subbands}.$$

Synthesis:

$$\text{subbands} \rightarrow \hat{x}[n].$$

4. Perfect reconstruction requires two things

Alias cancellation:

$$F_0(z)H_0(-z) + F_1(z)H_1(-z) = 0.$$

Desired-path condition:

$$\frac{1}{2}[F_0(z)H_0(z) + F_1(z)H_1(z)] = cz^{-d}.$$

5. Filter banks are a natural bridge to ML

They can be:

- fixed,
- learned,
- or hybrid.

The deck ends with exactly this synthesis.

Exit Question

> What is the key computational idea shared by efficient decimation and efficient interpolation?

Weak answer

> Both use polyphase filters.

True, but incomplete.

Good answer

Both reorganize filtering so that expensive operations occur at the lower sampling rate.

Very good answer

Polyphase structure avoids calculations that cannot affect the final useful output. In decimation, it avoids computing outputs that will be discarded. In interpolation, it avoids multiplying FIR coefficients by inserted zeros.

Excellent answer

Both exploit rate-change structure to move filtering to low-rate branches, while preserving exactly the same multirate input-output relationship.

That is probably the strongest conceptual answer.

Additional Concept Questions

Question 1

For $L = 5$, which output indices are produced by phase $k = 3$?

Good answer

$$y[5m + 3].$$

Therefore the indices are

$$3,8,13,18, \ldots$$

Question 2

Does interpolation polyphase processing remove the need for an anti-imaging filter?

Good answer

No. It only implements the same interpolation filter more efficiently.

Excellent answer

Polyphase changes the computation schedule, not the interpolation design requirements. The filter still must suppress spectral images and have the required gain.

Question 3

Why can't we move any arbitrary filter before an upsampler?

Good answer

> The Second Noble Identity requires the high-rate filter to have the special form $G(z^L)$.

Question 4

What does critical sampling mean in a two-channel filter bank?

Good answer

> Two subbands are each sampled at half the original rate, so the total output sample rate equals the original sample rate.

That is,

$$\frac{F_s}{2} + \frac{F_s}{2} = F_s.$$

Question 5

Does a critically sampled filter bank automatically guarantee perfect reconstruction?

Good answer

> No. Critical sampling only describes the total sampling rate. Perfect reconstruction additionally requires alias cancellation and an undistorted desired path.

This is an excellent conceptual distinction.

Question 6

What is the difference between a perfect-reconstruction filter bank and an ML feature bank?

Good answer

> A PR bank is designed so the original signal can be reconstructed exactly up to scale and delay. An ML feature bank is designed to preserve task-relevant information, so exact waveform reconstruction may not matter.

Three Misconceptions Worth Explicitly Correcting

Misconception 1

> “Upsampling increases the information in the signal.”

No.

Zero insertion does not create new information.

It changes the sample-rate representation.

The interpolation filter then constructs intermediate sample values consistent with the desired bandlimited representation.

Misconception 2

> “Polyphase interpolation works because zeros are ignored after filtering.”

No.

A better statement is:

> The polyphase structure avoids creating and processing those zero-valued samples in the first place.

Misconception 3

> “Filter banks and CNN channels are mathematically the same.”

No.

They share a structural analogy.

Classical filter banks usually come from frequency partitioning and multirate theory.

CNN feature channels are learned representations and may not correspond to recognizable physical frequency bands.

One Strong Numerical Example for Class

Suppose

$$L = 3$$

and

$$h[n] = \{1,2,3,4,5,6\}.$$

Then the phases are

$$e_0 = \{1,4\},$$

$$e_1 = \{2,5\},$$

$$e_2 = \{3,6\}.$$

For an input

$$x[n] = \{1,2,1\},$$

compute each branch.

Phase 0:

$$e_0 * x = \{1,6,9,4\}.$$

Phase 1:

$$e_1 * x = \{2,9,12,5\}.$$

Phase 2:

$$e_2 * x = \{3,12,15,6\}.$$

Then interleave:

$$y[0] = 1,$$

$$y[1] = 2,$$

$$y[2] = 3,$$

$$y[3] = 6,$$

$$y[4] = 9,$$

$$y[5] = 12,$$

and so on.

This gives students a concrete sense that each polyphase branch really does produce one residue class of output indices.

What Students Should Remember Most

If they leave the lecture remembering only four things, I would make them these:

1. Interpolation polyphase processing eliminates zero-valued multiplications.

2. The Second Noble Identity only works for the special $G(z^L)$ structure.

3. A two-channel filter bank is an analysis/synthesis system, not merely two unrelated filters.

4. Fixed and learned subband front ends are both ways of imposing structure on signal features, but only specially designed filter banks guarantee perfect reconstruction.

Week 7, Lectures 13 and 14, Exit Tickets

What are good exit ticket questions with answers for these lectures?

Gemini Flash Extended:

Lecture 13: Polyphase Decomposition & Efficient Decimation

Q1: Application of the First Noble Identity

Question: Why is it mathematically incorrect to commute a downsampler $\downarrow M$ directly in front of an arbitrary FIR filter $G(z)$ without using polyphase decomposition?

Answer: The First Noble Identity states that a filter and downsampler commute ($G(z^M) \rightarrow \downarrow M \equiv \downarrow M \rightarrow G(z)$) only when the filter possesses the specific z^M structural form (where impulse response taps are spaced by M samples). An arbitrary $G(z)$ operates on consecutive high-rate samples. Downsampling prior to filtering changes the input sequence fed into the taps, altering the effective system response and output values.

Q2: Polyphase Index Bookkeeping

Question: Given a 12-tap FIR filter $h[n]$ with indices $n \in \{0,1, \dots ,11\}$, list the specific tap indices n that belong to the phase branch $E_1(z)$ when using a decimation factor $M = 3$.

Answer: Polyphase branch k collects taps whose indices satisfy $n = rM + k$ ($n \pmod M = k$). For $M = 3$ and $k = 1$:

$$e_1[r] = h[3r + 1] \quad \text{for } r = 0,1,2,3$$

The corresponding tap indices are $n \in \{1,4,7,10\}$.

Q3: Origin of Decimation Arithmetic Savings

Question: Where does the M-fold computational savings in a polyphase decimator come from if the total number of FIR filter taps across all branches remains equal to N?

Answer: In a naive decimator, the FIR filter calculates every output sample at the high sampling rate F_s, only for the downsampler to immediately discard $M-1$ out of every M calculated samples. Polyphase decimation rearranges the system so that each branch filter $E_k(z)$ operates at the reduced output rate F_s/M. It computes strictly the retained output samples, reducing arithmetic from $\approx N \cdot F_s$ to $\approx \frac{N}{M} \cdot F_s$ operations per second.

Q4: Structural Analogy: Polyphase DSP vs. Grouped CNNs

Question: Briefly contrast what is being partitioned in a polyphase DSP filter versus a grouped convolution layer (`nn.Conv1d`) in PyTorch.

Answer:

- **Polyphase DSP:** Partitions time/tap indices of a single 1D filter modulo M to achieve exact multirate equivalence while moving calculations to a lower sample rate.
- **Grouped Conv1d:** Partitions feature channels into independent processing groups to reduce parameter counts and cross-channel connections. It is an architectural design choice with learned weights rather than an exact sample-rate reduction technique.

Lecture 14: Efficient Interpolation & Filter Banks

Q1: Polyphase Interpolation & Zero-Stuffing

Question: Why does naive interpolation waste arithmetic, and how does polyphase interpolation avoid this waste?

Answer: Naive interpolation inserts $L-1$ zeros between consecutive input samples ($\uparrow L$) and feeds the sequence into a full-rate FIR filter running at $L \cdot F_s$. The filter evaluates multiplications where $L-1$ out of L input samples are known zeros. Polyphase interpolation applies the Second Noble Identity ($\uparrow L \rightarrow G(z^L) \equiv G(z) \rightarrow \uparrow L$) to evaluate L low-rate branch filters $E_k(z)$ directly on non-zero input samples at rate F_s, interleaving the outputs afterward. This eliminates zero-multiplications and yields an L-fold arithmetic savings.

Q2: Conditions for Perfect Reconstruction (PR)

Question: In a two-channel analysis/synthesis filter bank, what are the two distinct mathematical requirements needed to achieve Perfect Reconstruction ($\hat{x}[n] = c \cdot x[n-d]$)?

Answer:

1. **Alias Cancellation:** $F_0(z)H_0(-z) + F_1(z)H_1(-z) = 0$, which ensures that the spectral folding created by downsampling and upsampling in the subbands cancels out upon synthesis recombining.
2. **Distortion Elimination:** $\frac{1}{2}[F_0(z)H_0(z) + F_1(z)H_1(z)] = cz^{-d}$, which ensures the overall system transfer function simplifies to a pure scale factor c and linear delay d without frequency-dependent magnitude or phase distortion.

Q3: Trade-offs of Fixed vs. Learned Subband Front Ends

Question: Identify one advantage and one disadvantage of using a fixed DSP filter bank (e.g., Mel filter bank) as a feature extractor compared to a learned `Conv1d` front end in a machine learning classifier.

Answer:

- **Advantage of Fixed Bank:** Contains zero trainable parameters, requires less training data/compute, and provides stable, predictable spectral behavior.
- **Disadvantage of Fixed Bank:** Cannot adapt its subband boundaries or bandwidths to task-specific acoustic/signal variations, potentially discarding discriminative information.

Week 7, Lectures 13 and 14, Exit Tickets

What are good exit ticket questions with answers for these lectures?

ChatGPT: (Medium)

Here is a set of exit-ticket questions I would use for these two lectures. I would keep most of them short enough to answer in 1–3 minutes, but design them to reveal whether students understand the structure rather than just remember terminology.

For Lecture 13, the most important ideas are: polyphase decomposition groups taps by index modulo M, the First Noble Identity only applies with the required z^M structure, efficient decimation avoids computing outputs that will be discarded, and grouped convolution is only an analogy to polyphase structure.

Lecture 13 — Polyphase Decomposition and Efficient Decimation

1. Core conceptual question

Question: In one or two sentences, what problem does polyphase decimation solve?

Good answer: Polyphase decimation reorganizes the FIR computation so that we do not calculate full-rate filter outputs that the downsampler will immediately discard. It gives the same retained output samples with less arithmetic.

What a strong answer should include:

- same filter / same output,
- less wasted computation,
- filtering effectively occurs at the lower rate.

---

2. Polyphase indexing

Question: For $M = 5$, which FIR coefficients belong to phase E_3?

Answer:

$$h[3], h[8], h[13], h[18], \ldots$$

because

$$e_3[r] = h[5r + 3].$$

Common weak answer: "Every fifth coefficient."

That is incomplete because it does not identify the offset.

---

3. Meaning of the decomposition

Question: What does

$$H(z) = \sum_{k=0}^{M-1} z^{-k} E_k(z^M)$$

actually do to the original filter?

Good answer: It partitions the original FIR coefficients into M residue classes according to their indices modulo M. It does not change the coefficients or design a new filter; it is an exact rearrangement.

---

4. Computational savings

Question: A 60-tap FIR decimator downsamples by $M = 5$. Roughly how many FIR multiplications per input sample are required after an ideal polyphase reorganization?

Answer:

$$\frac{N}{M} = \frac{60}{5} = 12$$

multiplications per input sample on average.

Excellent answer: The retained output still depends on roughly all 60 taps, but outputs occur only once every five input samples, giving an average cost of about 12 multiplies per input sample.

---

5. First Noble Identity

Question: Why is the statement "a downsampler can always be moved before a filter" incorrect?

Good answer: The First Noble Identity only permits the move when the high-rate filter has the special form $G(z^M)$. An arbitrary $G(z)$ does not commute with $\downarrow M$.

This is one of the most important misconceptions identified in the lecture.

---

6. Explain the savings correctly

Question: Which statement is more accurate, and why?

A. Polyphase reduces the number of FIR coefficients from N to N/M.

B. Polyphase keeps roughly N coefficients but avoids evaluating unnecessary full-rate outputs.

Answer:

B.

Good explanation: The coefficients are distributed among M branches. The total number of coefficients is essentially unchanged. The savings come from performing the computation at the lower output rate rather than calculating samples that the downsampler discards.

---

7. Short numerical reasoning

Question: A 32-tap FIR is followed by $\downarrow 4$. Compare the approximate multiplication rates of the naive and efficient implementations.

Answer:

Naive:

$$32F_s$$

multiplies/s.

Polyphase:

$$\frac{32}{4}F_s = 8F_s$$

multiplies/s.

So the ideal savings are about

$$4\times.$$

---

8. ML connection

Question: What is one similarity and one important difference between polyphase decomposition and grouped `Conv1d`?

Good answer: Similarity: both divide a larger computation into structured independent branches.

Difference: polyphase partitions time/tap phases modulo M to preserve an exact multirate system, while grouped convolution partitions feature channels and restricts learned connectivity.

The deck explicitly warns that grouped convolution is an analogy, not an equivalent polyphase implementation.

---

9. Misconception check

Question: True or false: all branches in an efficient decimator simply receive the identical sequence `x[::M]`.

Answer: False.

Good explanation: The branches correspond to different sample phases and therefore require appropriate phase offsets/delays. They are not all fed the same decimated sequence.

---

10. Best Lecture 13 exit question

If you want only **one** question at the end of the class, I would use:

Question: An engineer says, "Polyphase decimation is faster because it replaces an N-tap FIR with an N/M-tap FIR." What is wrong with that explanation?

Excellent answer: The original N coefficients are not removed; they are split across M polyphase branches. The main saving comes because the branches run at the lower rate F_s/M, so outputs that would have been discarded are never computed.

That question probes both the computational argument and the decomposition itself.

For Lecture 14, the main ideas are: interpolation polyphase processing avoids zero-valued multiplies, the Second Noble Identity also requires a special structure, two-channel banks perform analysis and synthesis, perfect reconstruction requires alias cancellation plus an undistorted desired path, and fixed versus learned filter banks have different purposes.

Lecture 14 — Efficient Interpolation and Filter Banks

1. Core interpolation question

Question: What computational waste occurs in a naive interpolator?

Good answer: After upsampling by L, $L-1$ zeros are inserted between adjacent input samples. A full-rate FIR then performs many multiplications using those known zeros.

---

2. Polyphase interpolation

Question: What does the equation

$$y[mL+k] = (e_k * x)[m]$$

mean?

Good answer: Polyphase branch k computes the output samples whose indices are congruent to k modulo L.

For example, for $L = 4$, phase 2 produces

$$y[2], y[6], y[10], y[14], \ldots$$

The lecture uses this interpretation as the main intuitive explanation of the polyphase interpolator.

---

3. Second Noble Identity

Question: Complete the valid equivalence:

$$\uparrow L \rightarrow \boxed{} \equiv G(z) \rightarrow \uparrow L.$$

Answer:

$$G(z^L).$$

So

$$\uparrow L \rightarrow G(z^L) \equiv G(z) \rightarrow \uparrow L.$$

---

4. Noble Identity misconception

Question: Why can an arbitrary $G(z)$ not simply be moved before an upsampler?

Good answer: The Second Noble Identity requires the high-rate filter to have the special structure $G(z^L)$. Without delays spaced in multiples of L, the two systems do not produce the same sequence.

---

5. Arithmetic savings

Question: A 40-tap FIR interpolates by $L = 5$. What is the approximate ideal speedup from polyphase implementation?

Answer:

$$5 \times.$$

Naive:

$$40(5F_s) = 200F_s$$

multiplies/s.

Polyphase:

$$40F_s$$

multiplies/s.

---

6. Compare decimation and interpolation

Question: What is the key computational difference between the waste eliminated in efficient decimation and the waste eliminated in efficient interpolation?

Good answer: In decimation, polyphase processing avoids calculating filtered outputs that would later be discarded. In interpolation, it avoids multiplying filter coefficients by zero-valued samples inserted by the upsampler.

This is a particularly strong exit-ticket question because it links the two lectures directly. The lecture wrap-up emphasizes this distinction.

---

7. Critical sampling

Question: Why is a two-channel analysis bank with each branch downsampled by 2 called critically sampled?

Answer: Each branch has sample rate $F_s/2$, so the combined sample rate is

$$\frac{F_s}{2} + \frac{F_s}{2} = F_s.$$

Thus the total number of subband samples per second equals the number of original input samples per second.

---

8. Filter-before-decimate reasoning

Question: Why do the lowpass and highpass analysis branches filter before $\downarrow 2$?

Good answer: Each branch needs to restrict its bandwidth before reducing the sample rate. Otherwise spectral components can alias after downsampling.

---

9. Perfect reconstruction

Question: What does perfect reconstruction mean in a two-channel filter bank?

Good answer:

$$\hat{x}[n] = c\,x[n-d].$$

The reconstructed signal may differ from the original by a constant scale c and a pure delay d, but there should be no other distortion.

---

10. Two requirements for perfect reconstruction

Question: What are the two separate requirements for perfect reconstruction?

Answer:

1. Alias cancellation:

$$F_0(z)H_0(-z) + F_1(z)H_1(-z) = 0.$$

1. Undistorted desired signal path:

$$\frac{1}{2}[F_0(z)H_0(z) + F_1(z)H_1(z)] = cz^{-d}.$$

Good verbal answer: The synthesis bank must cancel the alias terms created by downsampling, and the remaining desired transfer function must be only a gain and delay.

---

11. Haar bank reasoning

Question: For the Haar bank,

$$a[m] = \frac{x[2m] + x[2m+1]}{\sqrt{2}},$$

$$d[m] = \frac{x[2m] - x[2m+1]}{\sqrt{2}},$$

what information does $a[m]$ approximately represent, and what does $d[m]$ represent?

Good answer: $a[m]$ is average-like and therefore represents low-frequency or slowly varying content. $d[m]$ is difference-like and represents high-frequency or rapidly changing detail.

---

12. Fixed vs. learned filter banks

Question: Give one advantage of a fixed DSP filter bank and one advantage of a learned `Conv1d` front end.

Good answer: A fixed filter bank is interpretable, predictable, and may require little training data. A learned front end can adapt its filters to the specific classification task.

---

13. Anti-aliasing in ML

Question: A neural network uses a `Conv1d` layer with stride 2. Does the fact that the filter is learned eliminate the need to think about aliasing?

Answer: No.

Good explanation: Stride 2 is still a form of downsampling. If the learned filter does not sufficiently suppress components above the reduced Nyquist limit, aliasing can still occur.

---

14. Perfect reconstruction vs. classification

Question: Does a learned feature bank used for classification need to satisfy perfect reconstruction?

Answer: Not necessarily.

Good explanation: For classification, the goal is to preserve task-relevant information rather than reconstruct the original waveform. Perfect reconstruction is required only if exact recovery of the signal is part of the system objective.

---

15. Best Lecture 14 exit question

If you want one high-value question, I would use:

Question: Explain why a polyphase interpolator is more efficient even though it implements exactly the same interpolation filter.

Excellent answer: Upsampling creates $L - 1$ zeros between input samples. A naive full-rate FIR multiplies many coefficients by those zeros. The polyphase form partitions the filter into L output phases, computes each phase directly at the low input rate, and then interleaves the results. Therefore it gives the same interpolated output while avoiding zero-valued multiplications.

A Good Two-Lecture Cumulative Exit Ticket

Because Lectures 13 and 14 form a very natural pair, I would also consider ending Lecture 14 with a cumulative question:

Question: Complete the following comparison.

	Efficient decimation	Efficient interpolation

	Efficient decimation	Efficient interpolation
Rate changer	$\downarrow M$	$\uparrow L$
Waste in naive implementation	?	?
What polyphase avoids	?	?
Approximate ideal savings	?	?

Good answer:

	Efficient decimation	Efficient interpolation
Rate changer	$\downarrow M$	$\uparrow L$
Waste in naive implementation	computing outputs later discarded	multiplying inserted zeros
What polyphase avoids	unnecessary full-rate output calculations	zero-valued products
Approximate ideal savings	M	L

That is probably my favorite **Week 7 exit ticket**, because students must organize both lectures around the underlying computational principle rather than memorize two separate algorithms.

Recommended Set for Actual Classroom Use

If you want only **three questions after each 75-minute lecture**, I would use these:

Lecture 13

1. For $M = 4$, identify the coefficients in phase E_2.
2. Explain where the approximately M-fold computational saving comes from.
3. Why can $\downarrow M$ not be moved through an arbitrary filter?

Lecture 14

1. Explain what $y[mL + k] = (e_k * x)[m]$ means.
2. What are the two requirements for perfect reconstruction?
3. Contrast the computational waste removed by polyphase decimation with that removed by polyphase interpolation.

Those six questions collectively test **indexing, computational reasoning, Noble identities, output-phase interpretation, filter-bank understanding, and the connection between the two lectures**, which is a stronger diagnostic than asking students to simply reproduce the polyphase equations.
